App/Web para circulo de lectura

2ª Entrega – BD y primeras funcionalidades

Descripción - Abstract

Este anteproyecto propone una aplicación web para gestionar y dinamizar un círculo de lectura local. La plataforma facilita la organización de actividades, la comunicación y el intercambio de opiniones entre los miembros, priorizando la accesibilidad, la seguridad y la adaptabilidad a distintos dispositivos. El proyecto integra el diseño de la base de datos, la interfaz y funcionalidades clave como la gestión de usuarios, eventos, listas y reseñas.

Gonzalo Lavado Jaen

gonzalolj@hotmail.es

0. Índice de Contenidos

Apartado	***Página***
0. Índice	1
1. Introducción	2
2. Objetivos del proyecto	2
3. Marco teórico	2
4. Desarrollo del proyecto	3
4.1. Anteproyecto	3
A. Esbozo de la BBDD	3
Modelo de Entidad Relación	4
Diagrama de Atributos	5
Esquema Funcionalidades Básicas	6
Recapitulación BBDD	7
B. Esbozo de la Interfaz Web	7
Página de Inicio/Bienvenida	8
Página de Buscador/Catálogo	9
Página de Listas	9
Página de una Lista	10
Página de un Libro	10
Página de Eventos	11
Página de un Evento	11
Página de Perfil	12
4.2 Selección de Tecnologías y Funcionamiento en Desarrollo	13
A) Selección de Tecnologías	13
B) Creación de la BD	13
C) Puesta en marcha de web/placeholder	14
D) Inicio de servidor NodeJs/Express	14
E) Primeras funcionalidades	16
Página de Detalle de un Libro	16
Implementación del Tema Oscuro/Claro	16
F) Próximos Pasos	17
5. Conclusiones	17
5.1 Primera Entrega: Anteproyecto	17
5.2 Segunda Entrega: Primeras funcionalidades y puesta en marcha	17
6. Bibliografía	18
7. Anexo I: Manual de usuario	19
8. Anexo II: Manual de instalación	19
9. Anexo III: Otros anexos	20
10. Anexo IV: Rutas de ejemplo	21

1. Introducción

En el presente anteproyecto de ***Trabajo Fin de Grado*** (en adelante ***TFG***) se aborda el diseño y desarrollo de una aplicación web destinada a la gestión y dinamización de un círculo de lectura local. La iniciativa surge de la necesidad de contar con una herramienta digital que facilite la organización y divulgación de actividades, el intercambio de opiniones y la comunicación entre los miembros del grupo, adaptándose a las nuevas formas de interacción que demanda la sociedad actual.

Este trabajo se enmarca en el ciclo formativo de grado superior en Desarrollo de Aplicaciones Web, y responde a los requisitos y objetivos académicos del mismo, como el uso de tecnologías modernas de desarrollo web y la implementación de buenas prácticas en seguridad y accesibilidad. A lo largo del proyecto se expondrán los fundamentos teóricos, el proceso de diseño, la implementación técnica y los resultados obtenidos, vinculando cada apartado con las competencias profesionales que se pretende adquirir y demostrar.

2. Objetivos del proyecto

El principal objetivo de este proyecto es desarrollar una aplicación web intuitiva y accesible que permita a los usuarios gestionar eficientemente las actividades de un círculo de lectura local; así como ofrecer un espacio para la expresión y critica de diversas obras literarias. Con ello, se pretende facilitar la coordinación de eventos de lecturas conjuntas, la creación de debates y la compartición de recursos literarios entre los miembros del grupo.

Además, el proyecto busca promover la participación y el sentido de comunidad, proporcionando herramientas que fomenten la interacción y el intercambio de ideas. Entre los objetivos específicos destacan:

- Implementar un sistema de registro de usuarios seguro.
- Desarrollar módulos para la organización de eventos temáticos y sesiones de lectura, y garantizar la adaptabilidad de la plataforma a distintos dispositivos y niveles de competencia digital.
- Gestionar una estructura de comentarios y reseñas para libros y actividades.

En relación con el proyecto educativo, se persiguen varios objetivos específicos que van más allá del desarrollo de la aplicación en sí. En primer lugar, se busca consolidar los conocimientos adquiridos en el ciclo formativo sobre el diseño de bases de datos relacionales, asegurando la correcta estructuración de entidades como libros, usuarios, eventos y listas, así como la gestión de sus relaciones y atributos, incluyendo funcionalidades de comentarios y calificaciones.

Otro objetivo fundamental es potenciar las competencias en comunicación efectiva y documentación técnica, preparando al estudiante para integrarse con éxito en equipos multidisciplinares dentro del ámbito profesional del desarrollo web; así como conocer y profundizar en las diversas etapas de desarrollo de un proyecto.

Por último, se busca reforzar el compromiso con la accesibilidad y la seguridad en el desarrollo de aplicaciones, garantizando que la plataforma sea inclusiva y proteja la información de sus usuarios, aspectos esenciales en el contexto educativo y profesional actual.

3. Marco teórico

El marco teórico de este proyecto se fundamenta en conceptos clave del desarrollo web, la gestión de información y la estructura de comunidades digitales. En primer lugar, se abordan los principios del diseño centrado en el usuario, que orientan la creación de interfaces accesibles y fáciles de usar para personas con distintos perfiles, priorizando la experiencia y la interacción intuitiva. Se consideran también las mejores prácticas en arquitectura de software y bases de datos relacionales, elementos esenciales para estructurar adecuadamente la información y asegurar la escalabilidad y el correcto funcionamiento del sistema.

Adicionalmente, se plantea la importancia de una metodología estructurada basada en el desarrollo por fases, que facilita la organización y ejecución del proyecto:

- 1ª parte o "*Anteproyecto*": Diseño relacional y esbozo de la interfaz web. Esta fase inicial comprende el modelado de la base de datos mediante diagramas entidad-relación, la definición de entidades, relaciones y atributos clave. Además, se realiza un boceto de la interfaz web, determinando pantallas o páginas principales, funcionalidades básicas y tipos de personas usuarias (por ejemplo, administradoras, moderadoras y lectoras), sentando las bases para el desarrollo posterior.

- 2ª parte: Creación de la base de datos y elección de herramientas. En este momento se lleva a cabo la implementación física de la base de datos definida en la fase anterior, empleando sistemas gestores como MySQL o PostgreSQL. Simultáneamente, se seleccionan los softwares, lenguajes y frameworks necesarios para el desarrollo web, considerando criterios de compatibilidad, eficiencia y seguridad.
- 3ª parte o "*Versión preliminar*" y semi-funcional. Aquí se desarrolla un prototipo que integra la estructura de datos, las primeras funcionalidades esenciales y una interfaz básica. Esta versión permite realizar pruebas iniciales, obtener retroalimentación y ajustar tanto aspectos técnicos como de usabilidad.
- 4ª parte: Entrega final/funcional. En la última etapa, se completa la aplicación, optimizando la interfaz, integrando todas las funcionalidades previstas y garantizando que el sistema sea estable, seguro y accesible.

Es fundamental que las distintas fases del desarrollo estén diseñadas para poder relacionarse y retroalimentarse de manera continua. De este modo, los avances o ajustes en una etapa pueden influir en las siguientes, permitiendo mejorar el producto global, anticipar y/o corregir posibles errores, y asegurar la coherencia entre los componentes del sistema (base de datos, lógica de negocio, interfaz y experiencia de persona usuaria).

Por último, se incluyen aspectos fundamentales relativos a la seguridad y la protección de datos, atendiendo a la normativa vigente y a la importancia de salvaguardar la privacidad de quienes utilizan la plataforma colaborativa.

4. Desarrollo del proyecto

Esta gran sección se desglosará en función de las diversas etapas de entrega. Por lo que irá creciendo durante las sucesivas entregas y limitándose en esta en particular, al *Punto 4.2 – Selección de Tecnologías y Funcionamiento en Desarrollo*.

4.1 Anteproyecto

La tarea que nos atañe en el presente trabajo es la relativa a los cimientos de nuestra aplicación. Por lo que empezaremos 1º creando una BBDD mediante un Diagrama E/R y su Diagrama de Atributos asociado, dividiéndolo en 2 para mayor claridad (véase más adelante).

Una vez identificadas las diversas entidades necesarias y de las cuales guardaremos información. Llega el momento de empezar a esbozar una interfaz gráfica que contenga páginas para cada entidad (página de libro, página de evento, …), de forma que podamos mostrar los datos almacenados de una forma coherente y clara. Así como el resto de las páginas principales (P. de bienvenida, P. de buscador/catalogo, …). En este momento, también empiezan a surgir las diversas funcionalidades requeridas para que el usuario se pueda relacionar con las entidades y su información (ver un libro, editar una lista, …).

En todo momento y en la medida de lo posible, nos centraremos en diseños modulares y reutilizables. Y también se recuerda que nos encontramos en fases preliminares, por lo que se tiene en cuenta que durante el proceso puede surgir la necesidad de incluir (o eliminar) más atributos a las entidades, así como incluir y/o modificar las páginas propuestas.

Esta posibilidad de cambio, casa muy bien con los diseños modulares, tanto a nivel de interfaz como de partes del código. Teniendo pues que modificar solo secciones individuales cuyos cambios se propagan al resto de componentes que los utilizan (Ej.: Se diseña una cabecera y se propaga a todas las páginas que la usan).

A. Esbozo de la BBDD

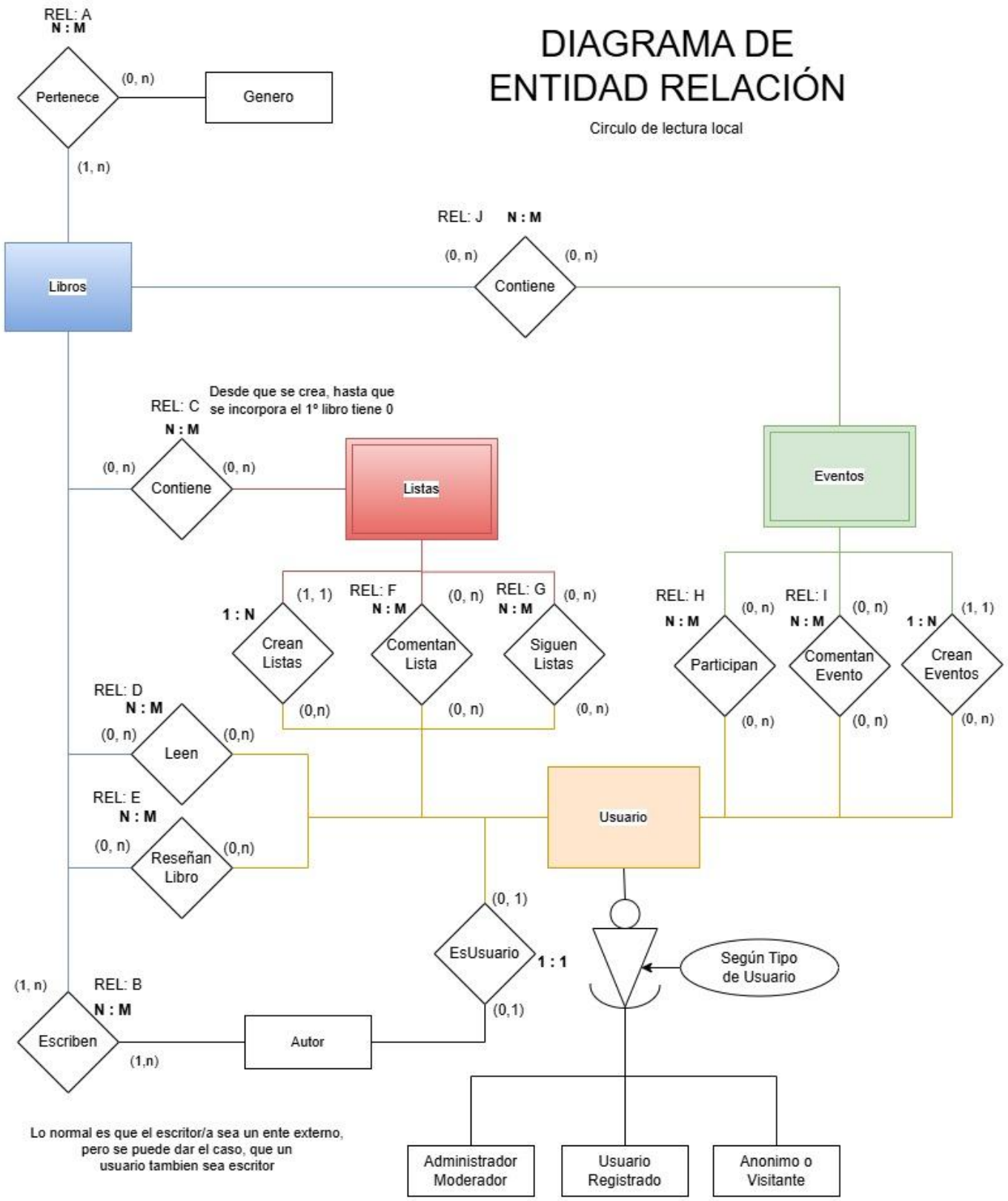
Para nuestra BBDD Principal (se sopesa aún en: si dividir en BBDD pública y privada o mantener una BBDD única) se propone un modelo relacional basado en las entidades fuertes "*Libro*" y "*Usuario*", en conjunto con las entidades débiles "*Lista*" y "*Evento*". Ya que estás, estarían asociadas a un usuario creador y que generalmente incluirán como contenido colecciones de libros (con excepción de eventos, que pueden no incluir libros específicos).

También, se requerirán una serie de entidades fuertes secundarias o de apoyo, como pueden ser "*Género*" y "*Autor*", para evitar duplicidad de datos y mejorar las consultas con filtros sobre los mismos. Dando también lugar a que un libro pueda tener más de un género y/o autor mediante relaciones "*N:M*" con la entidad libro.

Entidades Fuertes: **Usuario** **Libro** **Género** **Autor**

Entidades Débiles: **Lista** **Evento**

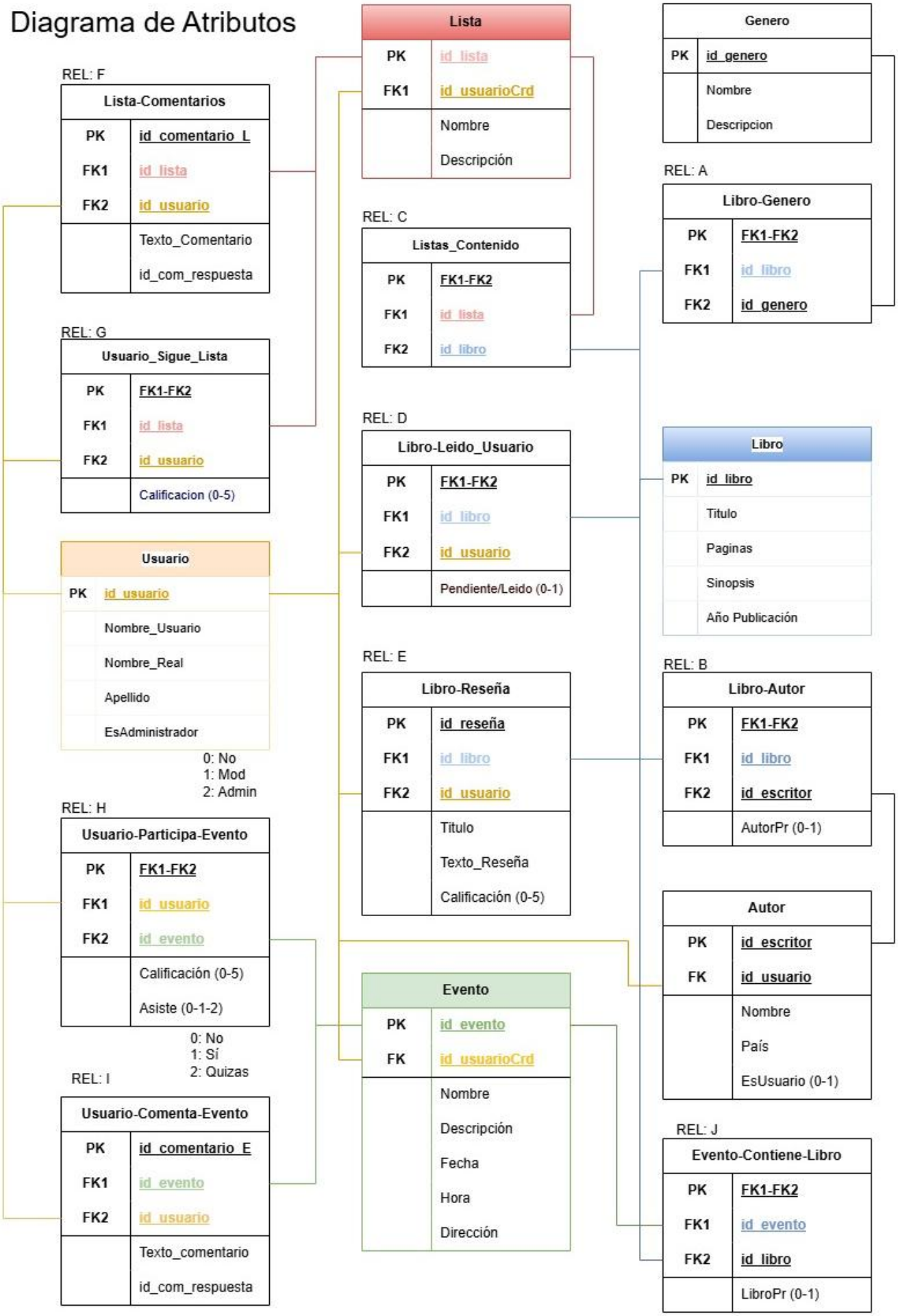
Modelo de Entidad Relación



A las relaciones “N:M” que implican una tabla con atributos, se les da nombre para una mejor identificación.

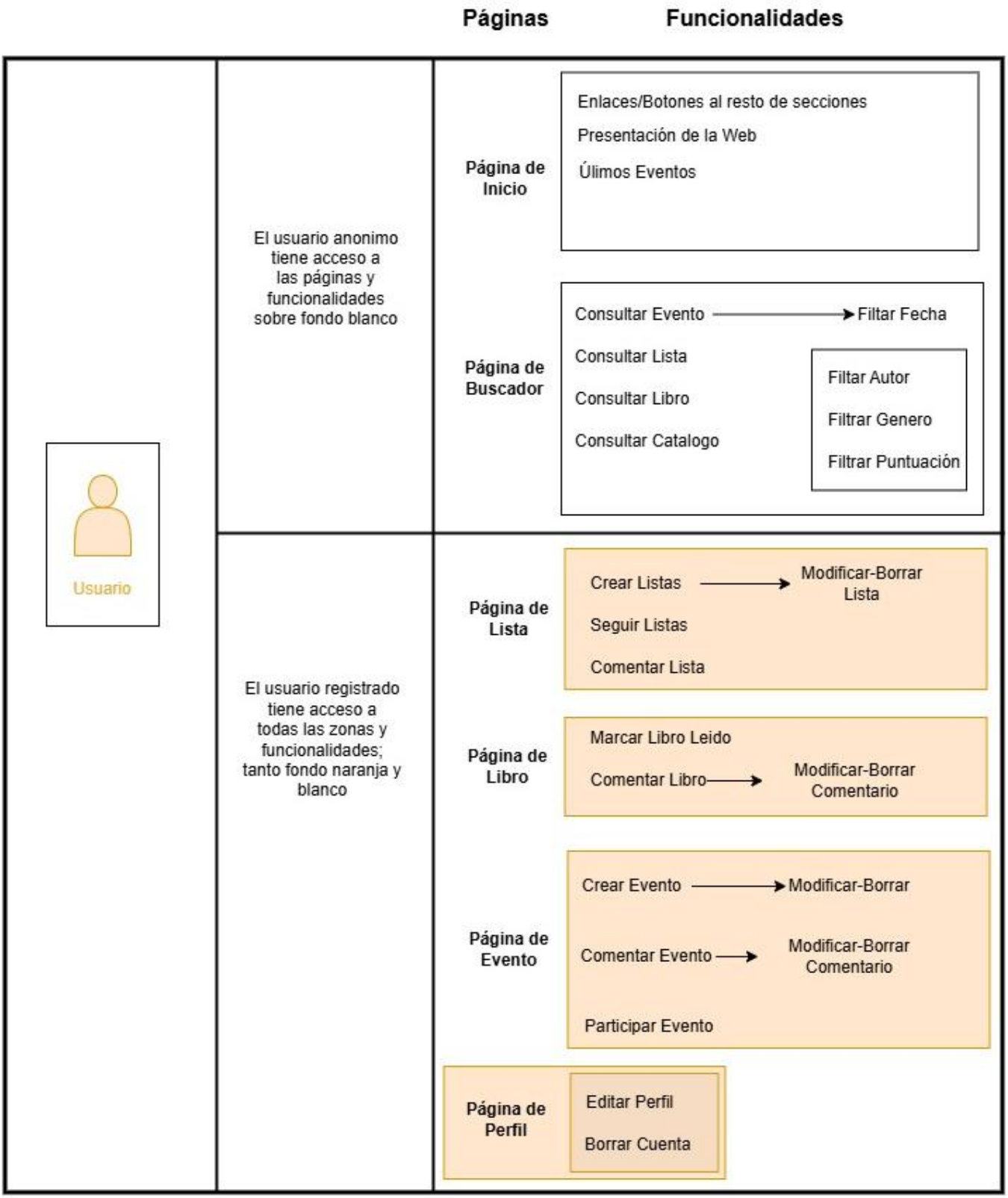
También, se menciona en el modelo, que habrá distintos tipos de usuarios (Visitante, Registrado y Administrador), no teniendo mayores implicaciones hasta fases posteriores. Pero que hay que tener en cuenta desde ya.

Diagrama de Atributos



Esquema Funcionalidades Básicas

Esquema de Funcionalidades



Recapitulación BBDD

Los modelos presentados, han sido realizados con el software de "draw.io"; y se aportan como datos adjuntos en formato PNG para mayor resolución y consulta.

Tenemos un modelo robusto con las entidades y sus relaciones bien identificadas; además, del mismo hemos deducido las páginas básicas que debemos diseñar, así como las diversas funcionalidades que deben poder realizarse desde esas páginas. Por lo que nos encontramos en condiciones de pasar al siguiente paso, y realizar un primer esbozo de la interfaz gráfica que vamos a requerir.

B. Esbozo de la Interfaz Web

Por ahora, se proponen las siguientes páginas principales:

- *Página de Inicio/Bienvenida*
- *Página de Buscador/Catálogo*
- *Página de Listas*
- *Página de Eventos*
- *Página de Perfil*
- *Página de un Libro*
- *Página de una Lista*
- *Página de un Evento*

Quedando aún lugar para páginas o secciones más secundarias (Inicio de sesión, edición de datos personales, …), que dependerán más de comportamientos y/o funcionalidades específicas. Por los que no se les dedicará un análisis previo en este momento, dejando su diseño y desarrollo para etapas posteriores.

Hay que mencionar que se opta, a modo personal; la selección por defecto de una paleta de colores de fondo oscuro, ya que considero que suele ser mejor para la vista y su fatiga. Quedando para más adelante la posibilidad de incluir una paleta de fondo claro como característica opcional; ya que, gracias a la composición modular, el cambio entre paletas se centra más en "*la elección de paletas de contraste suficiente y estética agradable*", más que a la implementación del cambio en sí.

También se acepta (*y se demanda*) cualquier sugerencia y/o mejora sobre las paletas elegidas.

De forma preliminar al desarrollo de las páginas en sí, se procede al diseño de los componentes compartidos, que en nuestro caso van a ser la "***Cabecera***" y el "***Pie***" de página. Atendiendo pues al requisito autoimpuesto de "*Diseño Modular*". Por lo que ayudándonos de la herramienta "Lunacy", procedemos a crear una serie de componentes compartidos:

Componentes Simples:

- Logo (Imagen y Nombre)
- NavBar (Botonera páginas)
- Cuadro de búsqueda (InputText+Imagen)

Componentes Compuestos:

+ *Cabecera – Default:*

- Logo
- NavBar
- Cuadro de búsqueda
- Fondo y Bordes Cabecera

+ *Cabecera – Sin Sesión*:

- *Cabecera – Default*
- Botones "*Registrarse*" e "*Iniciar Sesión*"

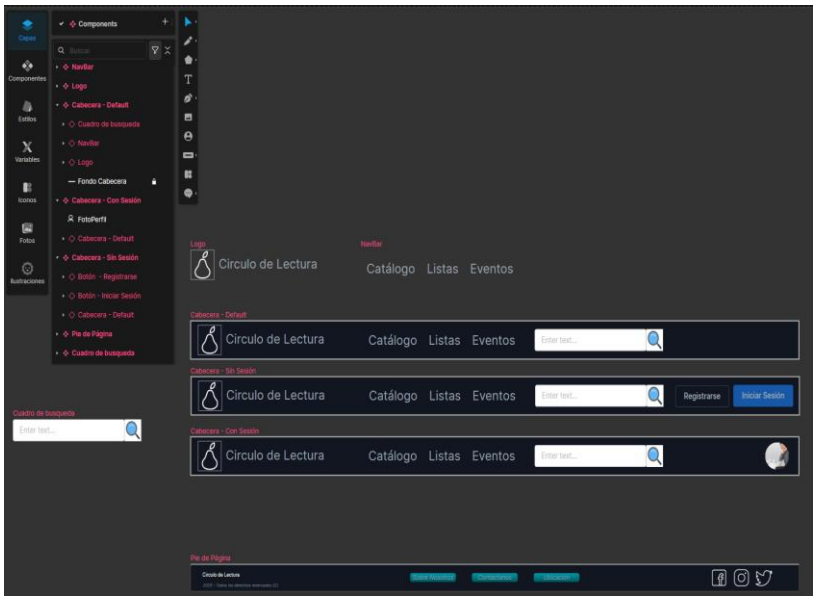


Imagen nº 1: Creación de componentes Simples y Compuestos (Lunacy)

+ *Cabecera – Con Sesión*:
    - *Cabecera – Default*
    - Icono "*Foto Perfil*"

+ *Pie de página*:
    - Mensaje Legal
    - Enlaces Internos
    - Iconos Redes
    - Fondo y Bordes Pie

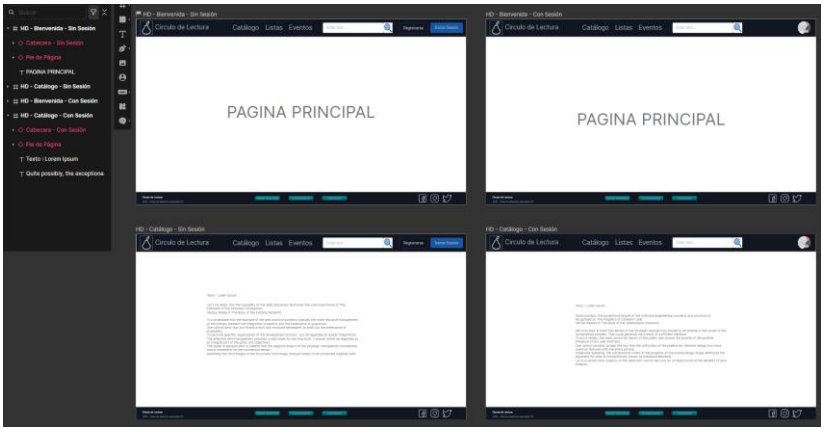


Imagen nº 2: Creación de Frames mediante componentes (Lunacy)

En el desplegable/explorador superior izquierdo de *Lunacy* (*Img.1*), podemos observar lo que se expone del diseño modular mediante componentes:

Por ejemplo, el componente "*Cabecera – Con Sesión*" se conforma únicamente con 2 elementos ("*Foto Perfil*" y "*Cabecera – Default*"); por lo que si editamos el componente padre (*Default*), los cambios se replicarán (o propagarán) a todos los elementos que lo usen. Por lo que con una simple edición en "Default", podemos modificar simultáneamente los componentes "*Con Sesión*" y "*Sin Sesión*". En un futuro, este diseño modular será igualmente replicado en la construcción efectiva de la aplicación (muy posiblemente con Angular19 y sus propios componentes nativos). También podemos ver el mismo comportamiento en el diseño preliminar de frames (*Img.2*).

Una vez tenemos el marco conceptual y el propio marco de los diversos frames; junto con una introducción a la metodología modular, pasamos a presentar el resultado del diseño para las diversas páginas, comentando alguna particularidad si procede, pero sin enumerar uno a uno el resto de componentes.

Para mayor detalle en cuanto al diseño, consultar el propio documento en la nube (requiere instalar el software).

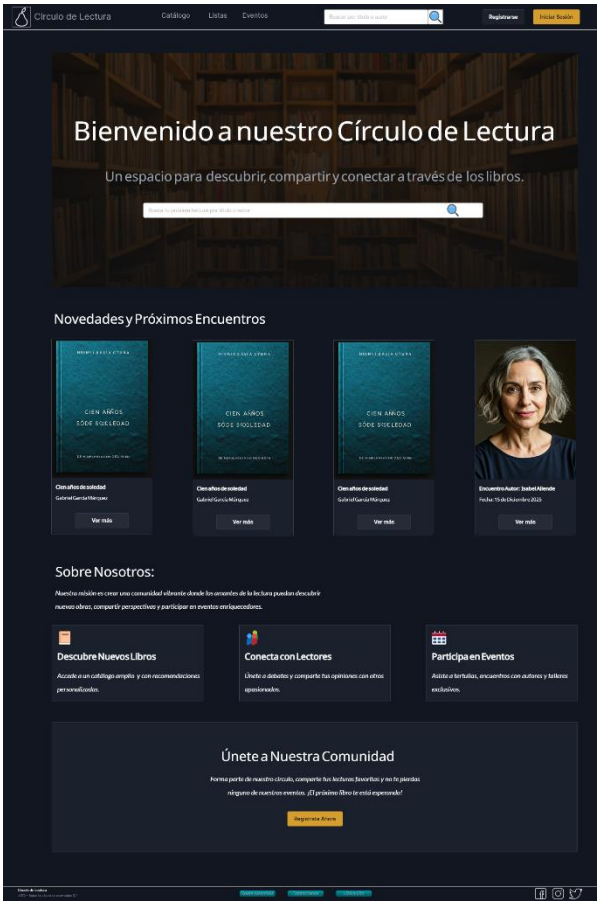
pulsar para ver documento en la nube

Página de Inicio/Bienvenida

Para la página de inicio, se opta por una página con scroll amplio que ofrezca una visión global del contenido.

Manteniendo el foco en la búsqueda de algún libro o evento con una imagen cálida y acogedora entorno a la lectura

Prácticamente todos los elementos son algún tipo de link a páginas de contenido sin necesidad de sesión (catalogo, eventos, listas, …)



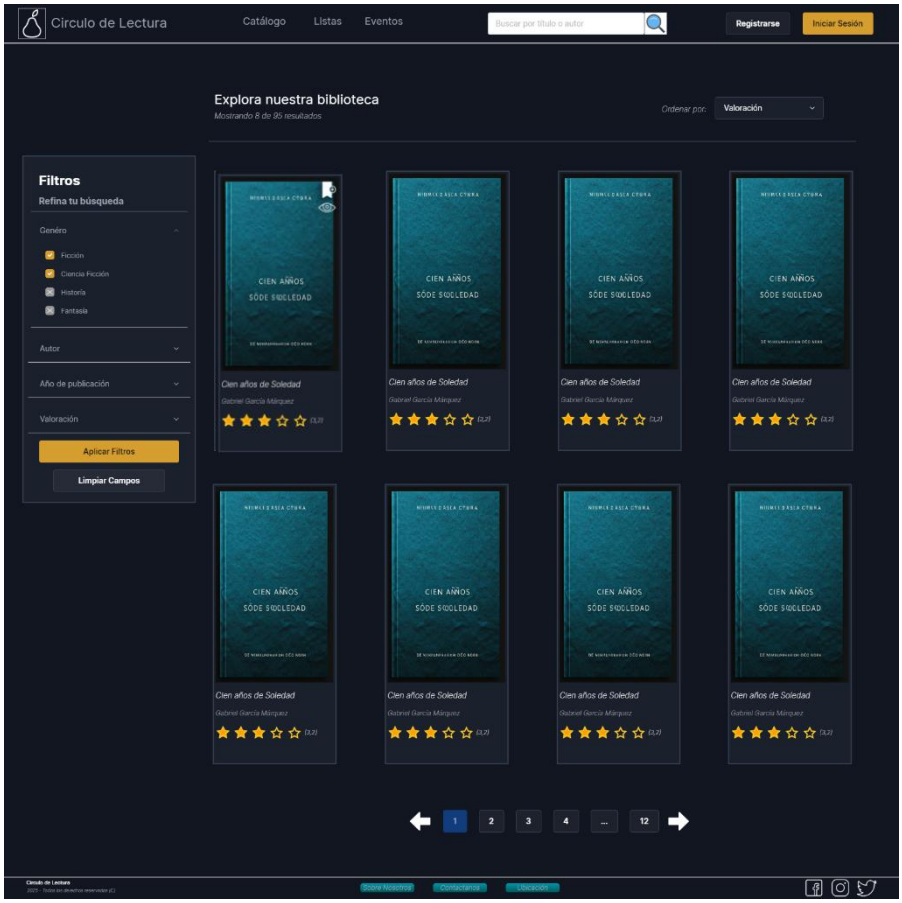
Página de Buscador/Catálogo

Para el catálogo de búsquedas se opta por un elemento "*aside*" para los diversos filtros. Utilizando desplegables para los diversos campos a filtrar.

También se opta por un diseño de cartillas en gridd para los elementos libro.

En las cartillas podemos encontrar imagen, titulo y autor de este. Así como la media de valoraciones. Además, si estamos con el cursor encima de la cartilla (*hover*), aparecerán iconos para marcar como visto y para agregar a alguna lista.

Se menciona como posibilidad futura, una segunda vista del contenido, con las cartillas en *flexrow* en lugar de *gridd*.

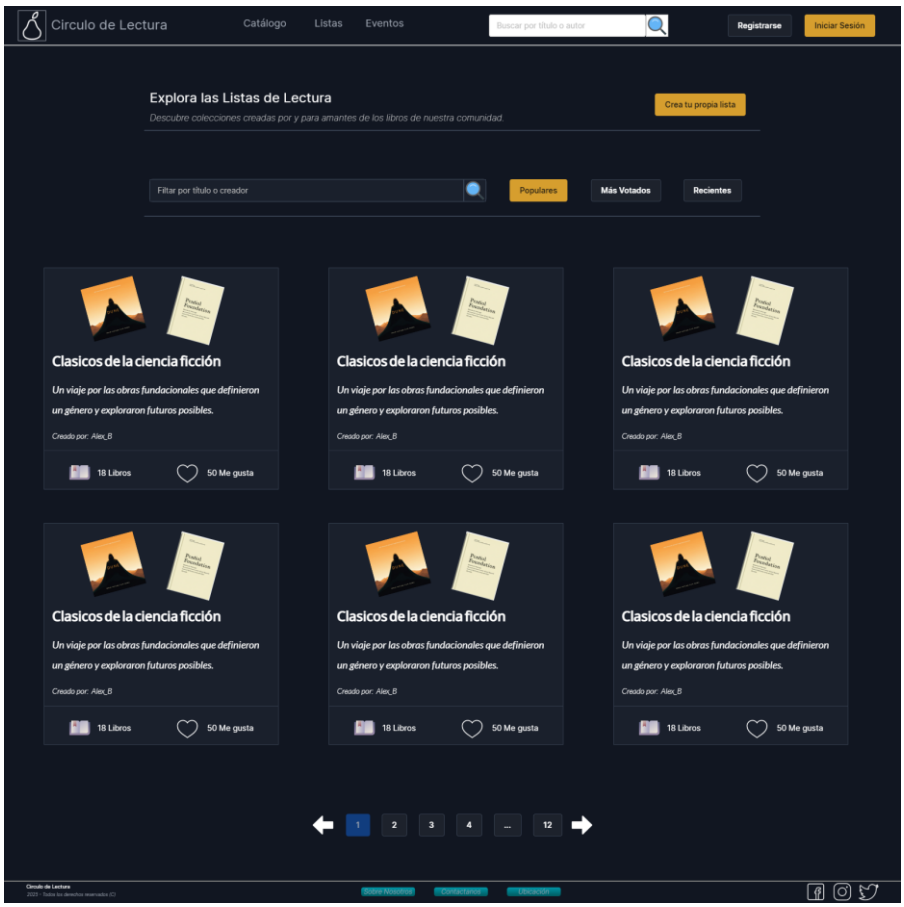


Página de Listas

En este caso, se opta de nuevo por una estructura *flexbox*, al no ser necesario un menú lateral (pocos filtros respecto a catalogo libros), no es necesario dividir el contendor principal en 2 secciones *gridd*, pudiendo implementar el cuerpo *flex* con el mosaico de tarjetas de lista directamente tras el encabezado de la sección (título y buscador especifico).

En cuanto a los filtros, estos se relacionan con:

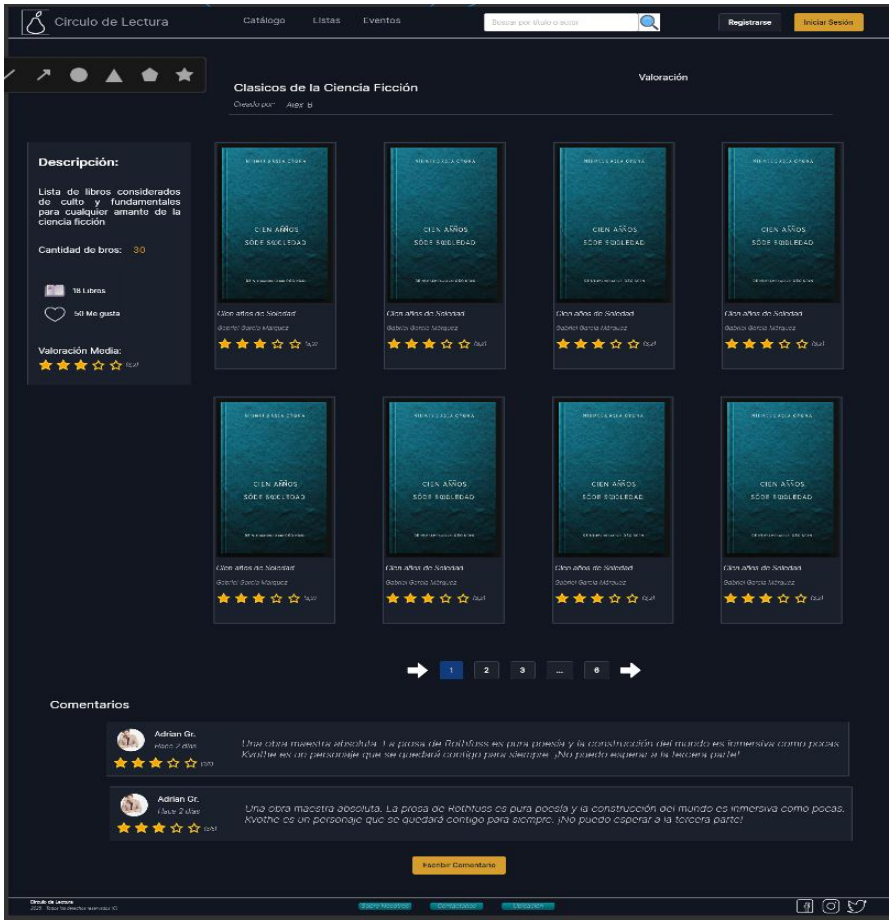
- Populares => Mayor nº de "Comentarios"
- Más Votados => Mayor nº de "Me gusta"
- Recientes => Última creada



Página de una Lista

Para la página de una lista, se reutiliza la disposición y componentes del catálogo general, logrando la misma estructura y distribución, con la reutilización de componentes.

Los cambios relevantes, son la inclusión de información descriptiva y estadísticas en el *sidebar*, así como la inclusión de una zona de comentarios.

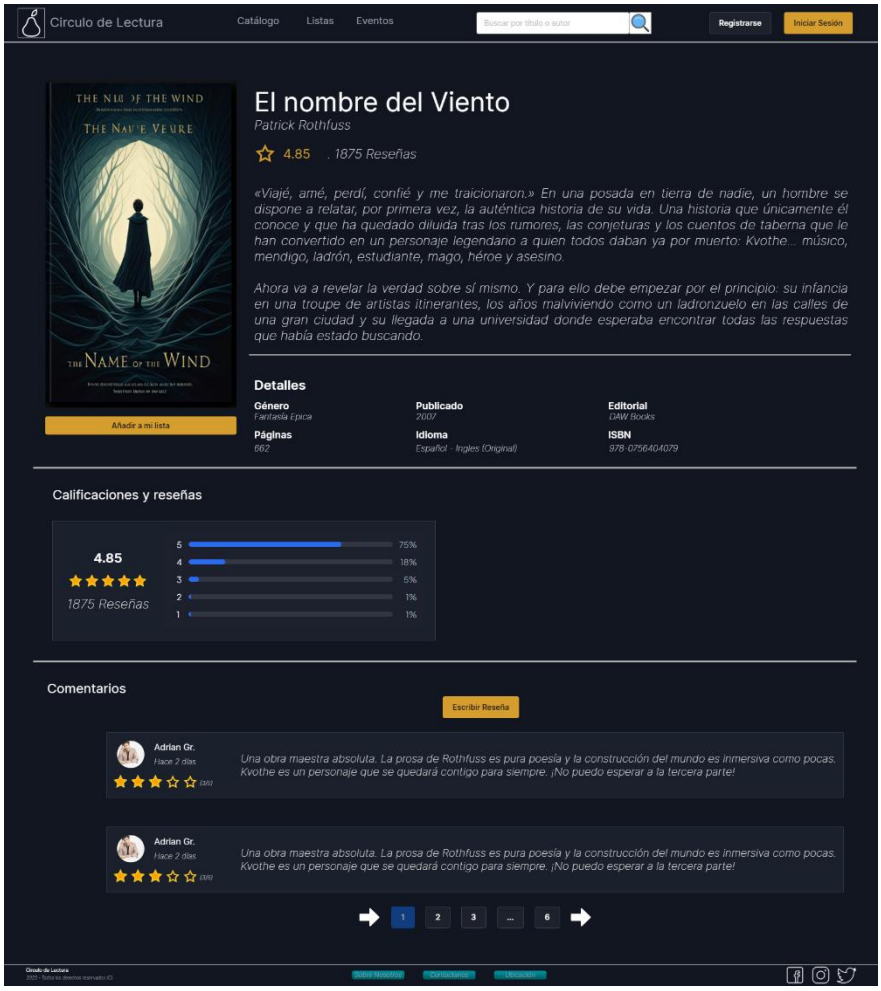


Página de un Libro

La página de un libro sigue la misma línea de diseño. En este caso, el contenido principal se centra en la presentación detallada del libro seleccionado, mostrando la portada, el título, el autor, la sinopsis y otros datos relevantes (como género, año de publicación o número de páginas).

Adicionalmente, se incorpora una sección para incluir información adicional como valoraciones medias, número de reseñas.

Debajo, se habilita una sección de comentarios donde los usuarios pueden dejar sus opiniones o preguntas sobre el libro, fomentando así la interacción y el intercambio de experiencias entre lectores.

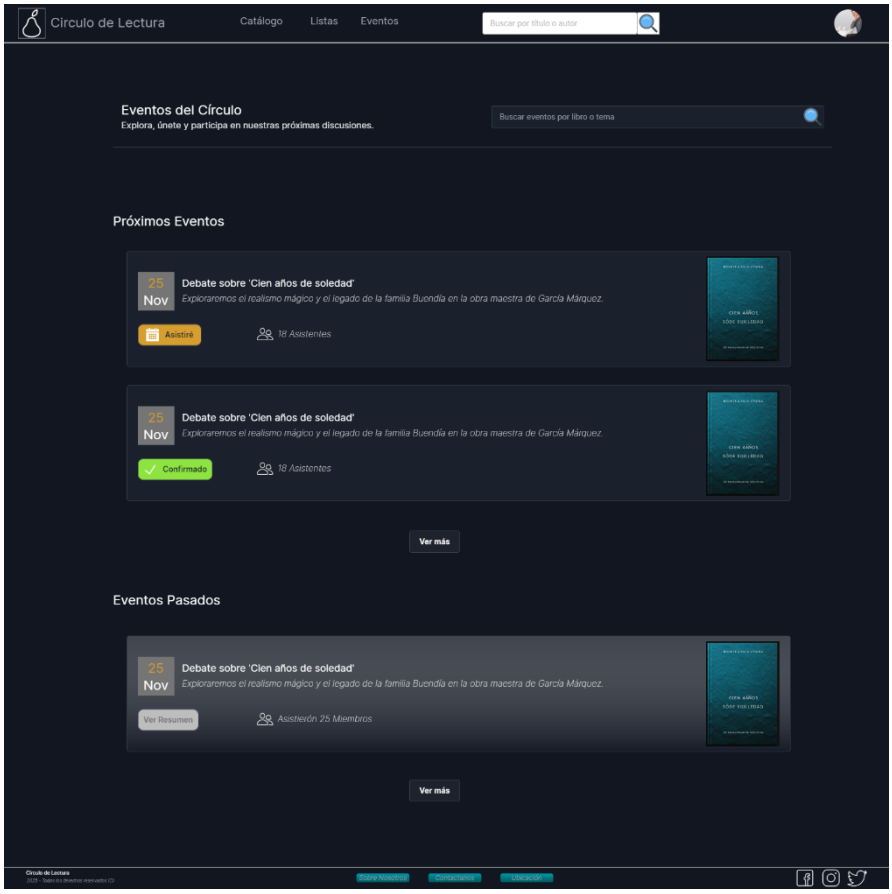


Página de Eventos

En este caso se opta por mostrar de forma separada los eventos existentes entre próximos y pasados. Reutilizando el mismo modulo de contenido con cambios de estado (degradado).

También se ofrece el poder desplegar las secciones y poder ver más eventos de cada tipo. Mostrándose los más próximos y los más recientes en primer lugar de forma respectiva.

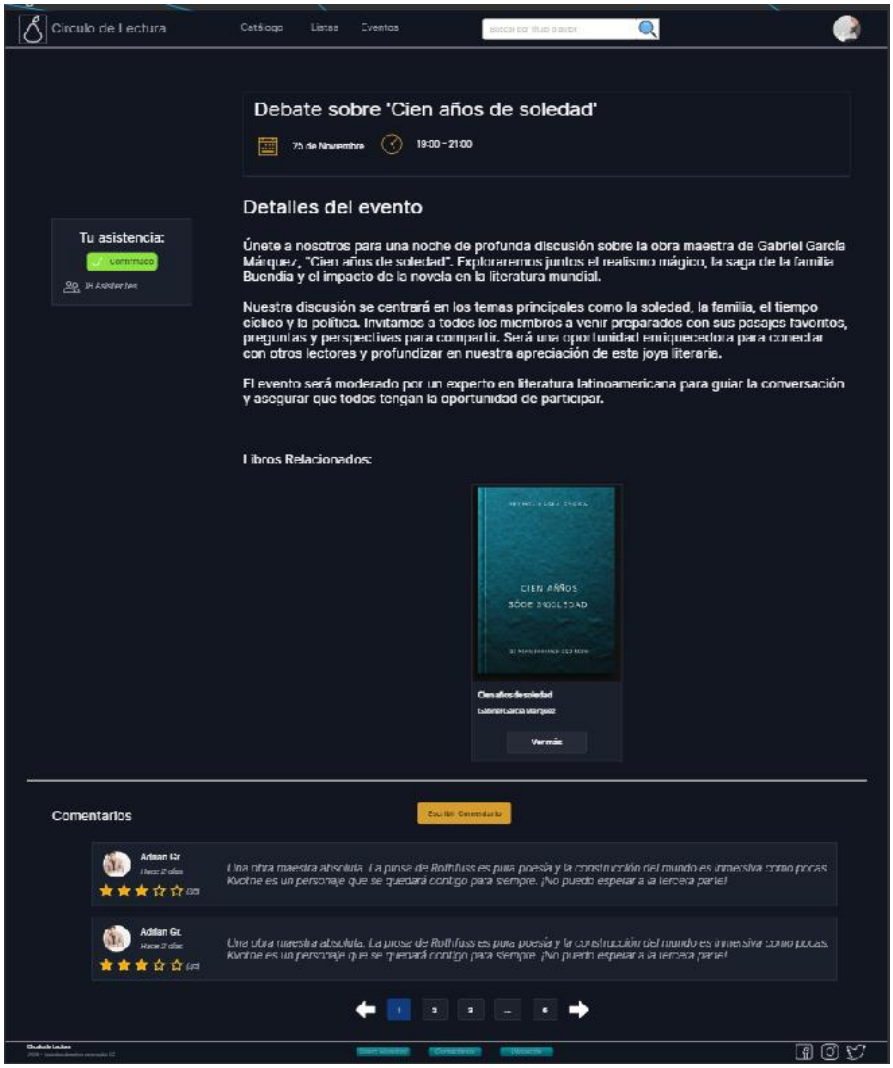
En este caso, para que se puedan mostrar los valores "*confirmado*" o "*has asistido*", en algún evento, se requiere de una sesión de usuario activa, por lo que se aprovecha para presentar un *frame* con "*Cabecera – Con Sesión*".



Página de un Evento

Para un evento en particular, tendremos una página simple y directa con la información relacionada y nuevamente a continuación (abajo) una zona de comentarios.

En este caso el *sidebar* se personaliza con la asistencia indicada con la sesión y con la posibilidad de expandir y desplegar un listado de asistentes.

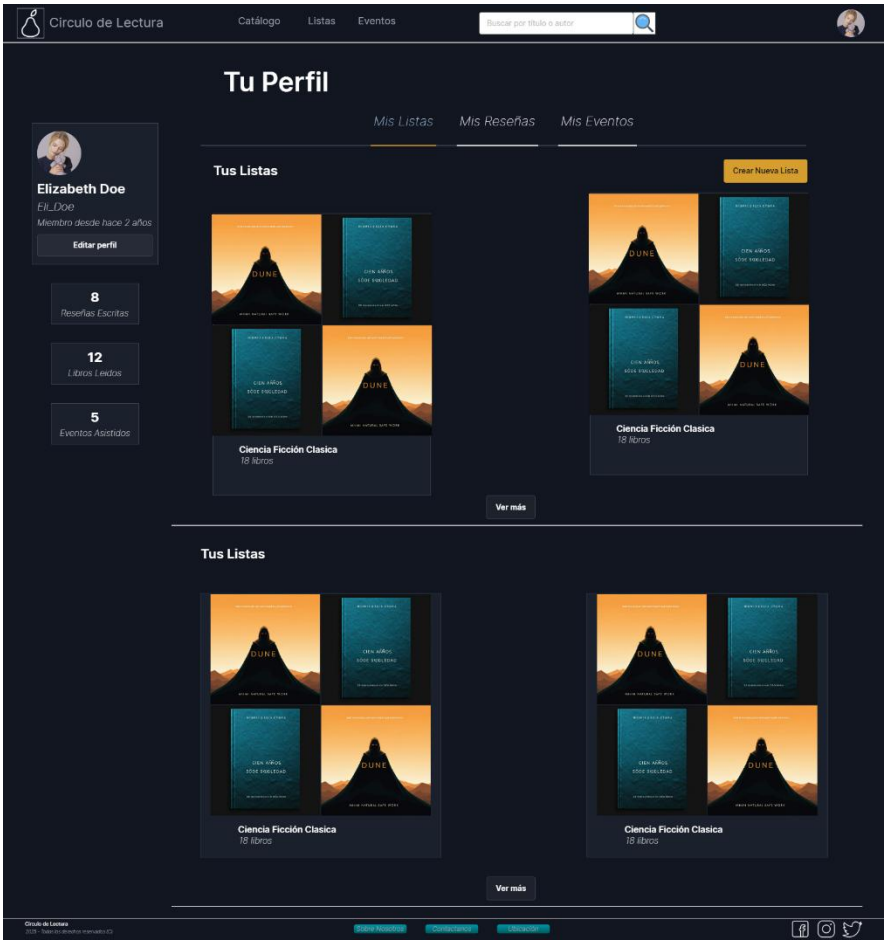


Página de Perfil

La página de perfil se subdividirá en 3, según el usuario quiera ver sus listas, reseñas o eventos. Ofreciendo para ello un *swich* de contenido mediante una botonera en el título de la página. Y resaltando la sección actual (en este caso *Listas*).

La disposición de las 3 secciones será la misma:

Presentación de las diversas tarjetas de elementos (lista, libro reseñado, evento asistido) mediante una estructura *flexrow* que se vaya repartiendo y ocupando el espacio existente (1º a lo ancho y luego saltando a otra fila si procede).



4.2 Selección de Tecnologías y Funcionamiento en Desarrollo

El presente apartado aborda la selección de tecnologías y el funcionamiento durante la fase de desarrollo del proyecto. Se describirán las herramientas, lenguajes y plataformas elegidos, así como los criterios que han guiado estas decisiones. Además, se detallará el proceso de implementación y las primeras pruebas realizadas, proporcionando una visión global del ecosistema técnico en el que se enmarca la solución propuesta.

A) Selección de Tecnologías

Actualmente, el sistema se implementa sobre una arquitectura de máquina única, en la que se ejecutan instancias separadas para cada uno de sus principales componentes: base de datos, backend y frontend. Esta configuración facilita la gestión y el aislamiento de procesos, permitiendo un desarrollo modular y controlado, adecuado para la fase inicial y el entorno de pruebas.

Para la gestión de datos se ha elegido MySQL como sistema de base de datos. MySQL destaca por su robustez, eficiencia y amplia compatibilidad, lo que asegura un almacenamiento fiable y estructurado de la información. Además, su integración con NodeJS resulta sencilla, garantizando una comunicación fluida entre la capa de datos y el backend.

El backend se desarrolla mediante NodeJS junto con el framework Express. Esta combinación permite construir APIs escalables, rápidas y flexibles, con especial atención a la gestión eficiente de solicitudes y la estructuración clara de rutas y controladores. NodeJS, al ser orientado a eventos y ejecutarse en un entorno de JavaScript, se adapta perfectamente a las necesidades dinámicas del proyecto.

En el frontend se ha optado por Angular 22, aprovechando su capacidad para desarrollar interfaces modernas y reactivas, así como su enfoque en la modularidad y reutilización de componentes. Angular facilita la creación de aplicaciones de usuario complejas manteniendo una estructura clara y mantenible, en línea con la propuesta de diseño basada en componentes, tal como se ha descrito en apartados anteriores.

Un aspecto clave de la arquitectura es la compatibilidad de TypeScript entre el frontend y el backend. El uso de TypeScript en ambos entornos permite compartir interfaces y modelos, reducir errores y mejorar la coherencia del código. Esta homogeneidad facilita la colaboración entre equipos y la integración de nuevas funcionalidades, asegurando que los datos y estructuras se mantengan consistentes a lo largo del desarrollo.

La construcción basada en componentes, tanto en Angular como en el backend, refuerza la propuesta de un diseño escalable y adaptable. Este enfoque permite que cada elemento (listas, eventos, reseñas) se represente como una unidad independiente, facilitando la personalización y el mantenimiento. Relacionando estos puntos con los conceptos expuestos en el documento, la arquitectura elegida se alinea con la estrategia de flexibilidad, personalización y adaptación que se busca, garantizando una base técnica sólida para el crecimiento futuro del sistema.

B) Creación de la BD

Durante la fase de desarrollo, la base de datos inicial se ha dividido en dos scripts SQL diferenciados. El primero de ellos se encarga tanto de la destrucción como de la creación completa de la base de datos, incluyendo la definición de las tablas y sus relaciones. El segundo script contiene una población inicial y representativa de todas las tablas principales (libros, autores, géneros, etc.), facilitando así las pruebas y la verificación del correcto funcionamiento del sistema desde el principio.

Cabe destacar que, a medida que avanzaba el desarrollo, se han introducido algunas modificaciones menores respecto al diseño propuesto originalmente. Por ejemplo, se ha añadido un campo de fecha con valor por defecto *NOW()* en las tablas de comentarios, con el objetivo de registrar automáticamente la fecha de inserción de cada entrada. Estos ajustes han surgido en respuesta a necesidades detectadas al trabajar con los datos reales y su integración con el backend, mejorando la usabilidad y la coherencia del sistema.

El modelo relacional actualizado, reflejando estos cambios; así como los scripts SQL; estarán disponibles y se mantendrán actualizados en el repositorio oficial del proyecto (Repositorio Proyecto TFC), lo que garantiza la trazabilidad y la consulta permanente de las últimas versiones del esquema de la base de datos.

Aunque hay que mencionar, que realmente, los scripts en uso se hayan en el repositorio correspondiente al backend, siendo los aquí aportados, meras copias para fines de presentación y documentación.

C) Puesta en marcha de web/placeholder

Para ilustrar la estructura del proyecto, se incluye una imagen detallada con la disposición de carpetas y el contenido relevante de cada una, mostrando claramente cómo se distribuyen los componentes, servicios y demás elementos clave en este momento.

Para la puesta en marcha inicial de la web se ha recurrido al esbozo en Lunacy, lo que permite generar con facilidad archivos HTML como placeholders iniciales. Estos archivos sirven como base desde la que se pueden extraer y adaptar componentes reutilizables, agilizando el proceso de prototipado y facilitando la integración posterior en el entorno Angular.

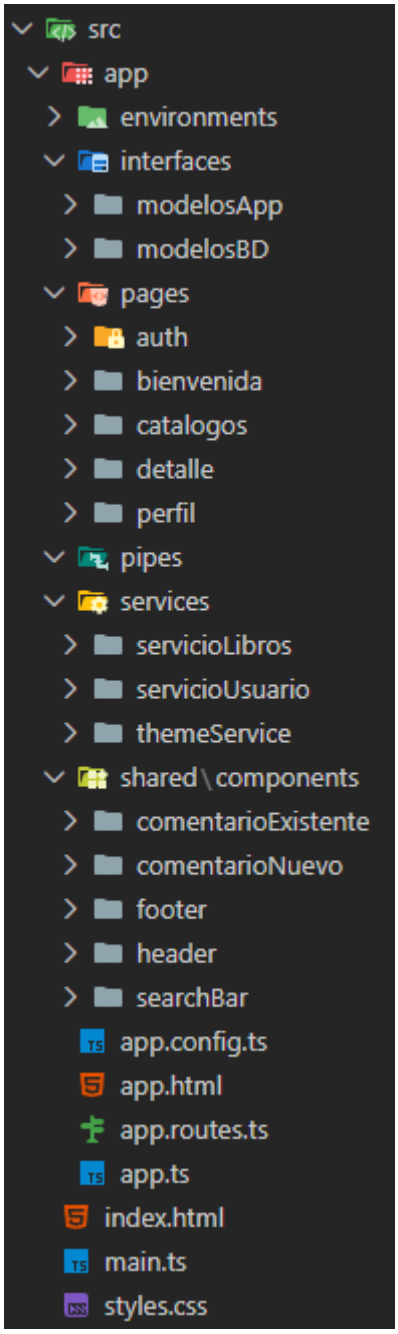
En cuanto a la definición de estilos, se emplea Tailwind para asignar clases directamente sobre los elementos HTML, limitando el uso de archivos CSS convencionales a casos muy específicos. Esta metodología permite una definición clara y compacta de cada componente, favoreciendo la legibilidad y el mantenimiento del código. Tailwind aporta flexibilidad y rapidez a la hora de ajustar las interfaces, al tiempo que reduce la dispersión de estilos y la necesidad de hojas de estilo extensas.

La estructura del proyecto se organiza siguiendo los principios de Angular, donde la modularidad es fundamental. Gracias a los componentes, servicios y guards propios del framework, cada funcionalidad se encapsula y se distribuye en carpetas específicas, por ejemplo: Pages/PaginaCatalogoLibros:

Pages/PaginaCatalogoEventos, entre otras. Esta organización permite una gestión más eficiente del código, facilita la reutilización de componentes y la integración de nuevas funcionalidades, y mejora la escalabilidad del sistema. Los servicios centralizan la lógica de negocio y el acceso a datos, mientras que los guards aseguran la protección de rutas y la gestión de permisos, contribuyendo a una arquitectura robusta y segura.

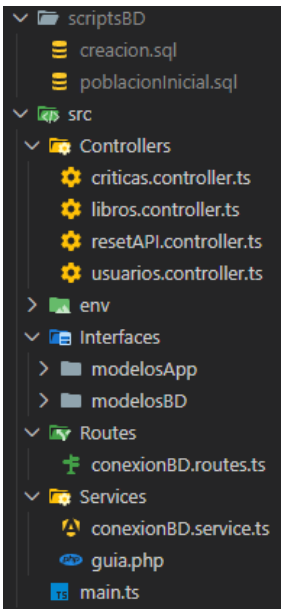
Para esta entrega en concreto, dada la necesidad de que los diversos componentes requieren datos de la base de datos. Se han realizado los básicos que no dependen de la misma, como son el Header, Footer y Search Bar (ya que además de en el Header aparece en más sitios).

Por otro lado, al no contar aún con una funcionalidad de usuarioIdentificado, se opta por recrear la experiencia sin identificación (no hay página de perfil ni posibilidad de crear libros o eventos por ejemplo; aunque la funcionalidad y rutaAPI existan).



D) Inicio de servidor NodeJs/Express

El backend se construye sobre NodeJs, utilizando Express para la gestión de rutas y respuestas HTTP. Se proporciona igualmente una captura de la estructura.



Entre las dependencias principales destacan:

- **cors**: Habilita el intercambio seguro de recursos entre dominios; actualmente para acceso externo en red Local.
- **dotenv**: Permite cargar variables de entorno desde archivos .env sin exponerlas públicamente.
- **Express**: Framework principal para la creación de servidores y gestión de rutas.
- **mysql2**: Cliente para conectar y operar con bases de datos MySQL.

En cuanto a las devDependencies, se emplean:

- @types/cors / @types/express / @types/node: Para el soporte de tipado TypeScript.
- typescript: Para añadir tipado estático y mejorar la calidad del código.

Se implementa una clase centralizada llamada **ConexionBD**, que encapsula las operaciones CRUD básicas sobre la base de datos. Además, ConexionBD incluye métodos específicos para consultas avanzadas, como *innerJoin* entre las tablas de libro, autor y comentario, facilitando la obtención de datos relacionados en una sola petición. Esta estructura modular permite mantener el código limpio y reutilizable, favoreciendo la escalabilidad del proyecto.

```
/** Actualizar registros de la tabla especificada que cumplan con las condiciones
 *
 * @param tabla String con el nombre de la tabla donde se actualizarán los registr
 * @param datos Objeto con los datos a actualizar, donde la clave es el nombre de
 * @param condiciones Objeto con las condiciones para actualizar los registros, do
 * @returns Número de registros afectados (actualizados).
 * @throws Error si ocurre algún problema durante la actualización o si los paráme
 */
async zactualizarRegistro(
    tabla: string,
    datos: Record<string, string | number | boolean | Date>,
    condiciones: Record<string, string | number>,
): Promise<number> {
```

Se muestra un ejemplo del método actualizarRegistro().

NodeJs permite definir respuestas personalizadas para cada ruta API, gestionando la redirección o prohibición de rutas no listadas mediante controladores. Cada ruta se asocia a un controlador que administra los métodos CRUD correspondientes, garantizando un flujo claro y seguro de datos. Durante el desarrollo, se habilita una ruta específica para la creación o reinicio de la base de datos, utilizada para pruebas y descartar errores iniciales, pero eliminada en producción para evitar riesgos innecesarios.

```
// Creación del router Express para manejar las rutas de la API
const conexionRouter = Router();

conexionRouter.get("/resetAPI", resetearAPI);

// Definición de las rutas para usuarios
conexionRouter.post("/usuario", crearUsuario);
conexionRouter.get("/usuarios", obtenerUsuarios);
conexionRouter.put("/usuario/:id", actualizarUsuario);
conexionRouter.delete("/usuario/:id", borrarUsuario);

// Definicion de las rutas para libros
conexionRouter.post("/libro", crearLibro);
conexionRouter.get("/libro/:id", obtenerLibroId);
conexionRouter.get("/libros", obtenerLibros);
conexionRouter.put("/libro/:id", actualizarLibro);
conexionRouter.delete("/libro/:id", borrarLibro);

// Definición de rutas para criticas/reseñas de libros
conexionRouter.post("/libro/:id/critica", crearCritica);
conexionRouter.get("/libro/:id/criticas", obtenerCriticasLibro);
conexionRouter.put("/libro/:id/critica/:criticaId", actualizarCritica);
conexionRouter.delete("/libro/:id/critica/:criticaId", borrarCritica);

// Redirección/Respuesta de rutas no definidas
conexionRouter.use((_req, res) => {
    res.status(404).json({ error: "Ruta no encontrada" });
});
```

Es importante destacar la compatibilidad de TypeScript entre el frontend y el backend, aprovechando las mismas interfaces para el tipado de datos. Se diferencian las **interfacesBD** (orientadas a consultas directas sobre tablas en consultas a la BD) de las **interfacesAPP** (estructuras adaptadas a la lógica de la aplicación y la respuesta a devolver al Front), lo que mejora la comprobación de tipos y minimiza errores de integración. Sin embargo, este enfoque requiere actualizar las interfaces cada vez que se modifica alguna tabla, asegurando la coherencia en todo el sistema.

Cabe destacar que, en la mayoría de las rutas, vamos a encontrar 2 métodos get, uno en singular acompañado de un parámetro id, que debe devolver un único elemento. Y otra ruta en plural, para devolver un numero indeterminado de elementos en función de condiciones.

```
/**
 * Crear nueva crítica para un libro
 */
> async function crearCritica(req: Request, res: Response) {···
}

/**
 * Obtener críticas de un libro
 */
> async function obtenerCriticasLibro(req: Request, res: Response) {···
}

/**
 * Actualizar crítica de un libro
 */
> async function actualizarCritica(req: Request, res: Response) {···
}

/**
 * Borrar crítica de un libro
 */
> async function borrarCritica(req: Request, res: Response) {···
}
```

Finalmente se muestra un ejemplo de controlador, cada uno de estos metodos, procesa los datos de la petición del front, los filtra/limpia y valida (y rechaza con mensaje de error si procede); construyendo una peticion apropiada con el metodo correspondiente de la clase Conexión del Servicio Central.

La estructura genérica nos permite enviar/construir condiciones tipo:

{campoColumna: “valor”} Que serán tratadas como un: Where campoColumna = valor

Y más complejas:

{campoColumna: {operador: “LIKE”, valor: “El Qui”}} Where campoColumna LIKE “%El Qui%”

El controlador prepara las condiciones que acepta o no de forma apropiada y las envía al método de Conexión General.

Se adjuntan una serie de rutas de ejemplo para pruebas en ANEXO 3 – OTROS ANEXOS

E) Primeras funcionalidades

Antes de que el backend estuviera completamente operativo, se realizó una etapa preliminar en el frontend centrada en la componetización. Esta fase se enfocó en la definición y estructuración de elementos estáticos clave, como el **header**, y en establecer la navegación funcional entre los distintos *placeholders* de la aplicación. De este modo, fue posible acceder a la página principal y comprobar que los componentes estáticos respondían correctamente y estaban bien integrados en la interfaz.

Página de Detalle de un Libro

Dado que prácticamente todas las páginas consumirán información procedente de la base de datos, se decidió comenzar el desarrollo funcional por la página de **detalleLibro**. Para facilitar las pruebas y asegurar la correcta integración de datos, se incorporó un botón temporal que permite alternar entre diferentes libros y verificar así la respuesta de los componentes. Aunque la mayoría de funcionalidades de los *controladores del Srv* aún se están puliendo con Postman.

La página de detalleLibro destaca por combinar la obtención y visualización de información relevante mediante un controlador que consulta varias tablas en la base de datos. Este controlador recopila datos principales del libro, información de los autores, géneros asociados, críticas y la nota media obtenida por el libro. Además, la estadística global de puntuaciones se desglosa mostrando los porcentajes de cada valoración (de 1 a 5 estrellas), lo que proporciona una visión detallada de la valoración del libro por parte de los usuarios. Esta estructura modular y la integración de múltiples fuentes de datos convierten la página de detalleLibro en la plantilla base sobre la que se desarrollarán futuras funcionalidades del proyecto.

A medida que avanza el desarrollo, van surgiendo nuevos componentes, como el formulario de escritura de un comentario (*comentarioNuevo*) y la visualización de comentarios existentes (*comentarioExistente*). Estos componentes son reutilizables y se adaptan al contenedor padre, lo que permite definir su estructura y tamaño en función de la página en la que se encuentren (por ejemplo, *detalleLista* o *detalleEvento*). Así, el uso de componentes aporta flexibilidad y eficiencia al desarrollo, facilitando la adaptación y reutilización según las necesidades de cada vista.

El desarrollo de la página de detalleLibro no solo supuso el primer consumo de datos reales del backend, sino que también fue el proceso más complejo en esta fase inicial, sirviendo de referencia y ejemplo para el resto de la aplicación. A través de su implementación, se definieron patrones de integración y componetización entre frontend y backend, sentando las bases para una arquitectura modular y reutilizable. Gracias a esta arquitectura, cada parte de la interfaz y los servicios (tanto en el frontend como en el backend) pueden evolucionar de forma independiente a partir de una base común —principalmente el método de *ConexionBD* y el formato adoptado para rutas y controladores—, lo que facilita el mantenimiento y la escalabilidad del proyecto.

Por último, queda pendiente resolver la gestión y almacenamiento de las imágenes de los libros. Actualmente se barajan dos alternativas principales: guardar las imágenes en MySQL como datos binarios o almacenarlas en un directorio del backend y guardar solo una referencia en la base de datos SQL. Mientras se toma la decisión definitiva, en la página se utiliza un *placeholder* como solución provisional para las imágenes de Libros y Avatar de Usuarios en los comentarios.

Implementación del Tema Oscuro/Claro

La funcionalidad de alternancia entre tema oscuro y claro se ha desarrollado como parte integral del servicio global asociado al **header** de la aplicación. En la práctica, esto significa que el botón correspondiente para cambiar el tema está disponible y puede ser pulsado desde cualquier página de la web, permitiendo así que la interfaz reaccione de manera dinámica a la interacción del usuario.

No obstante, cabe señalar que, aunque la lógica y la estructura para ambos temas están implementadas, únicamente se ha definido de forma explícita el tema oscuro. Las variables asociadas al tema claro han sido declaradas, pero permanecen sin valores asignados, es decir, están en blanco. Esta decisión responde a la dificultad personal que supone seleccionar una paleta estética adecuada para el modo claro.

Pese a esta limitación, el sistema funciona correctamente: el cambio de tema no genera errores, todas las páginas cargan y responden al evento de alternancia, y las variables del tema claro existen en el código, aunque con valores nulos. Por tanto, la funcionalidad puede considerarse operativa, quedando pendiente únicamente la definición de la paleta cromática específica para el modo claro.

F) Próximos Pasos

El siguiente paso en el desarrollo se centra en la página **catalogoLibros**, donde actualmente se está trabajando en la obtención del conjunto de libros y el pulido de la lógica de filtrado selectivo. En este momento, la tarea principal consiste en afinar los filtros para que respondan de manera precisa a las búsquedas y preferencias de los usuarios.

Mediante pruebas realizadas con Postman (anexando rutas de ejemplo para referencia), se ha comprobado que las funciones básicas de CRUD están operativas para los elementos principales: es posible crear, leer, actualizar y borrar una sola crítica, usuario o libro. Sin embargo, el filtrado complejo aún está en proceso de mejora para garantizar una experiencia más avanzada en la gestión de datos.

Una vez el catálogo de libros funcione correctamente con los filtros y se establezca la asociación efectiva a la página de detalle de cada libro (funcionalidad ya operativa), se podrá replicar la misma dinámica en el resto de secciones: página de bienvenida, catálogo y detalle de listas, y catálogo y detalle de eventos. Este enfoque modular permitirá un avance coherente y eficiente en la implementación de las funcionalidades básicas en todas las páginas.

Finalmente, quedará para el último lugar la integración del sistema de sesiones y el proceso de autenticación, ya que, por ahora, el acceso se realiza como usuario anónimo. Una vez completadas las funcionalidades principales, se abordará la inyección de sesiones y el desarrollo del mecanismo de login, garantizando la seguridad y personalización de la experiencia de usuario.

5. Conclusiones

5.1 Primera Entrega: Anteproyecto

Se ha presentado un esbozo de los estilos por defecto (distribución general de elementos con tipos *gridd* y *flex*, paletas y ubicaciones generales), atendiendo a tener una pantalla suficiente de al menos ***1600x*******. Pero se debe tener presente la posibilidad de adaptar los contenedores y distribuciones de elementos mediante las opciones de *mediaquerys* presentes en *HTML5* y *CSS3* para distintos tipos de pantallas según el tamaño disponible (móviles o tablets).

Esta tarea se facilitará al basarnos en un diseño vía componentes, ya que los contenedores y componentes se construirán con una fase de información proveniente del back, y se renderizarán según las reglas ***css*** habilitadas en el tipo de host que consume la parte *front*. Por lo que nuestra tarea de adaptar la distribución de contenidos en pantallas más pequeñas se facilitara en gran medida al particularizarse casi en su totalidad a establecer un tipo de ***css*** para cada tipo de pantalla mediante las *mediaquery* de ***CSS3***.

Por otro lado, este esbozo nos sirve como un plan general, pero aún no tiene carácter definitivo, puesto que según vayamos desarrollando e implementando las funcionalidades y contenido. Nos surgirán nuevas necesidades y/o encontraremos formas más "*estéticas*" de presentar el mismo contenido. Por lo que todo el esbozo debe considerarse como un punto de partida sobre el que ir construyendo y retroalimentando durante todo el desarrollo.

5.2 Segunda Entrega: Primeras funcionalidades y puesta en marcha

Durante esta segunda entrega se ha logrado avanzar significativamente en la puesta en marcha de las primeras funcionalidades de la aplicación, destacando especialmente la implementación del sistema de alternancia entre tema oscuro y claro. Aunque el modo claro aún requiere la definición de su paleta cromática, la estructura está preparada y la funcionalidad es plenamente operativa. Además, la disposición de los estilos y la adaptación a distintos dispositivos mediante *mediaquerys* refuerza la flexibilidad y escalabilidad del proyecto.

El desarrollo se ha apoyado en un enfoque modular mediante componentes, lo que facilita la gestión y evolución de la interfaz a medida que surgen nuevas necesidades. Este proceso iterativo permite considerar el diseño actual como un punto de partida, abierto a mejoras y ajustes según el feedback y la integración de nuevas funciones. Por tanto, la segunda entrega no solo consolida los avances técnicos, sino que evidencia una metodología de trabajo dinámica y adaptable, esencial para el éxito del proyecto en futuras fases.

6. Bibliografía

Actualmente, para el anteproyecto (Análisis previo y definiciones iniciales), la bibliografía y webgrafía utilizadas son principalmente los diversos contenidos y apuntes escolares; tanto del 1º curso (Diagramas E/R y BBDD's SQL) como del 2º curso (Interfaces y Entornos Servidor-Cliente). Por lo que no es posible/viable una exposición o referencia adecuada de los PDF's individuales o exactos utilizados.

Paralelamente, y concretamente para la parte del diseño de la interfaz básica, se han consultado y tomado como referencia diversas webs relacionadas o no con el mundo de la lectura. Pero que comparten funcionalidades y/o diseños interesantes para tomar como referencias, por lo que se procede a una enumeración de estas.

Catálogo de Webs consultadas:

Filmaffinity	(*BBDD audiovisual en español*)	https://www.filmaffinity.com/es/main.html
IMDB	(*Internet Movie Database*)	https://www.imdb.com/es-es/
Menéame	*(Agregador de noticias con comentarios)*	https://www.meneame.net/
Teslo-Shop	(*con versión didáctica propia que recreé en 1º*)	https://tesloshop-gc.vercel.app/
Wikipedia		https://es.wikipedia.org/wiki/Wikipedia:Portada
Asociación "*El Dirigible Rol Sevilla*"		https://app.eldirigible.es/

Fuente de rostros:

Generador de rostros por excelencia https://thispersondoesnotexist.com/

Software Empleado:

Diagramas:	draw.io
Diseño Gráfico:	lunacy
SQL:	MySql WorkBench 8.0
Desarrollo:	Visual Studio
Control de versiones:	Git/GitHub

7. Anexo I: Manual de usuario

Actualmente las únicas funcionalidades de usuario son:

- La navegación entre los diversos placeholders, pudiendo llegar a la página del libro con id 1

Catalogo => click en cualquier libro

- Botón temporal en la página de detalle de un libro, para variar el id y consultar las respuestas de comentarios y datos de los 40 libros existentes.

Apartado para expandir en entregas futuras. - WIP

8. Anexo II: Manual de instalación

Actualmente, debemos contar en una misma maquina con:

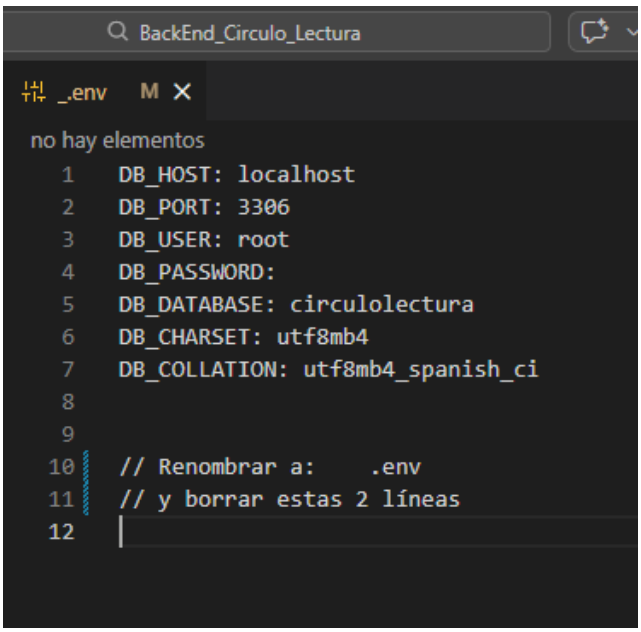
- Algún tipo de gestor de paquetes npm (por ejemplo, nvm o volta).
- Una instancia de mysql server (con o sin interfaz gráfica).
- Angular/CLI de forma global

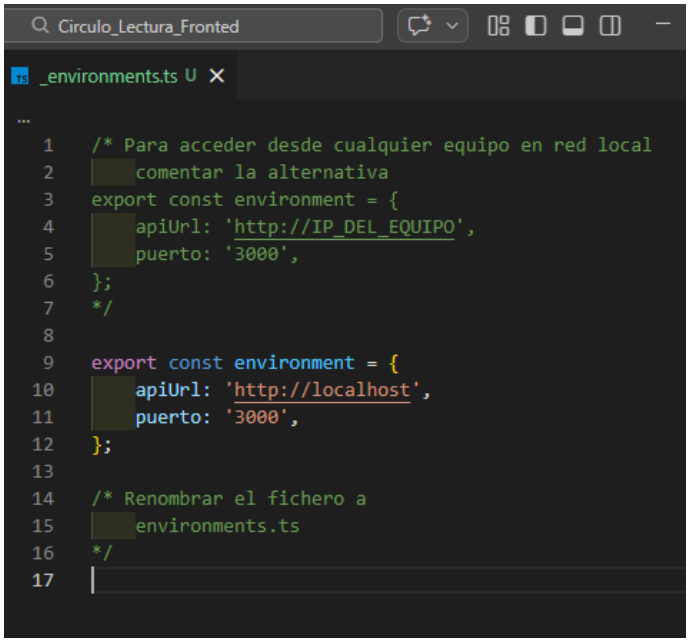
Instrucciones en readMe

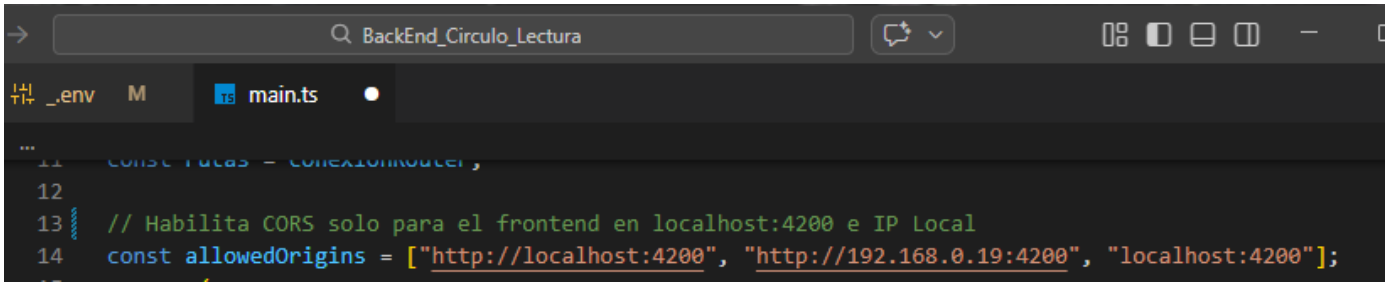
Tras esto, debemos clonar los 2 proyectos e iniciar una terminal en cada uno:

- Circulo_Lectura_Fronted
- Backend_Circulo_Lectura

En ambos proyectos, se deben editar los ficheros con variables de entorno según las instrucciones indicadas en cada uno.







En ambas terminales debemos realizar un:

npm install

Para iniciar el backend en su terminal ejecutamos:

npm run dev

Para iniciar o resetar la base de datos, realizar una petición get a la url (desde navegador o herramienta tipo Postman):

localhost:3000/resetApi

Para iniciar el fronted en su terminal ejecutamos:

ng serve

Si además queremos acceder desde un equipo/móvil externo, el comando anterior deberá ser:

ng serve --host 0.0.0.0

Apartado para expandir en entregas futuras. - WIP

9. Anexo III: Otros anexos

Enlaces de interés a documentación en otros formatos más visuales:

Esquemas del Modelo Relacional

Bocetos preliminares con Lunacyd

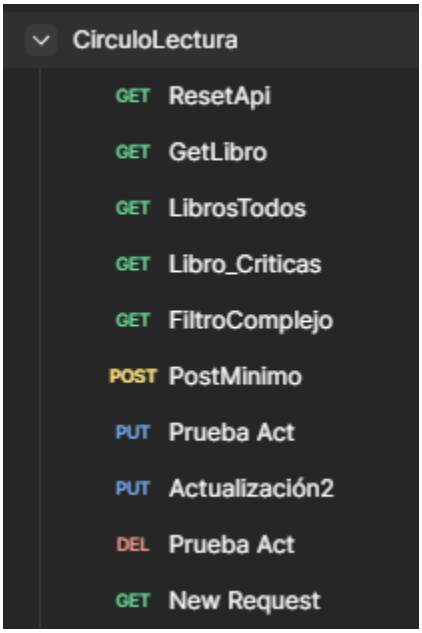
Enlaces a los repositorios específicos:

Repositorio: Fronted

Repositorio: Backend

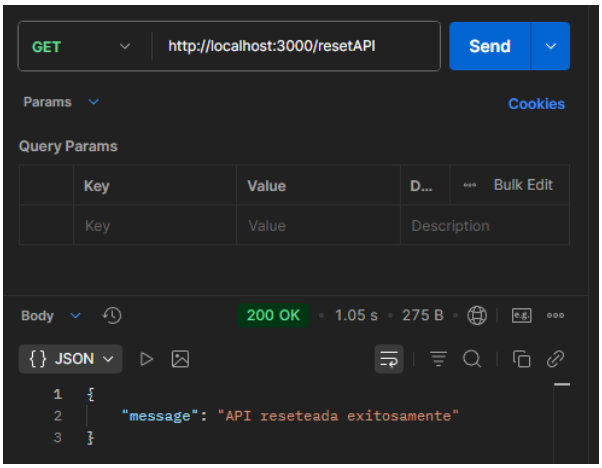
10. Anexo IV: Rutas de ejemplo:

Podemos consultar estar rutas/peticiones al Backend mediante navegador (Solo GET) o Postman (cualquier petición).

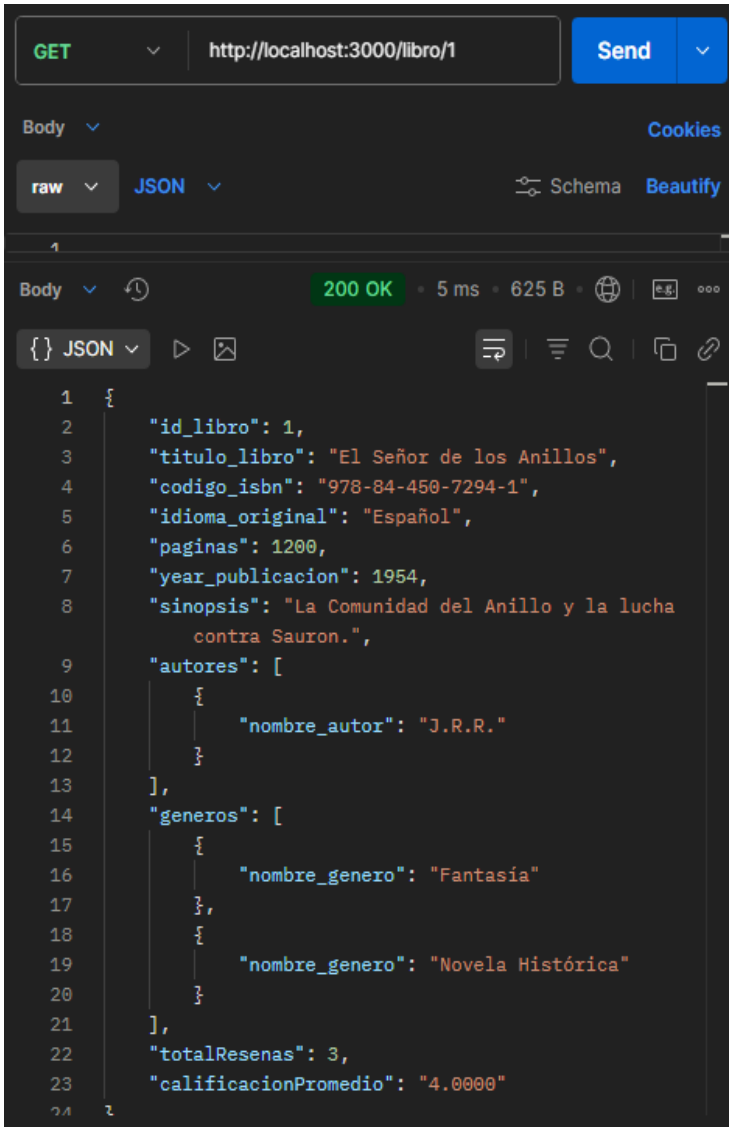


GET

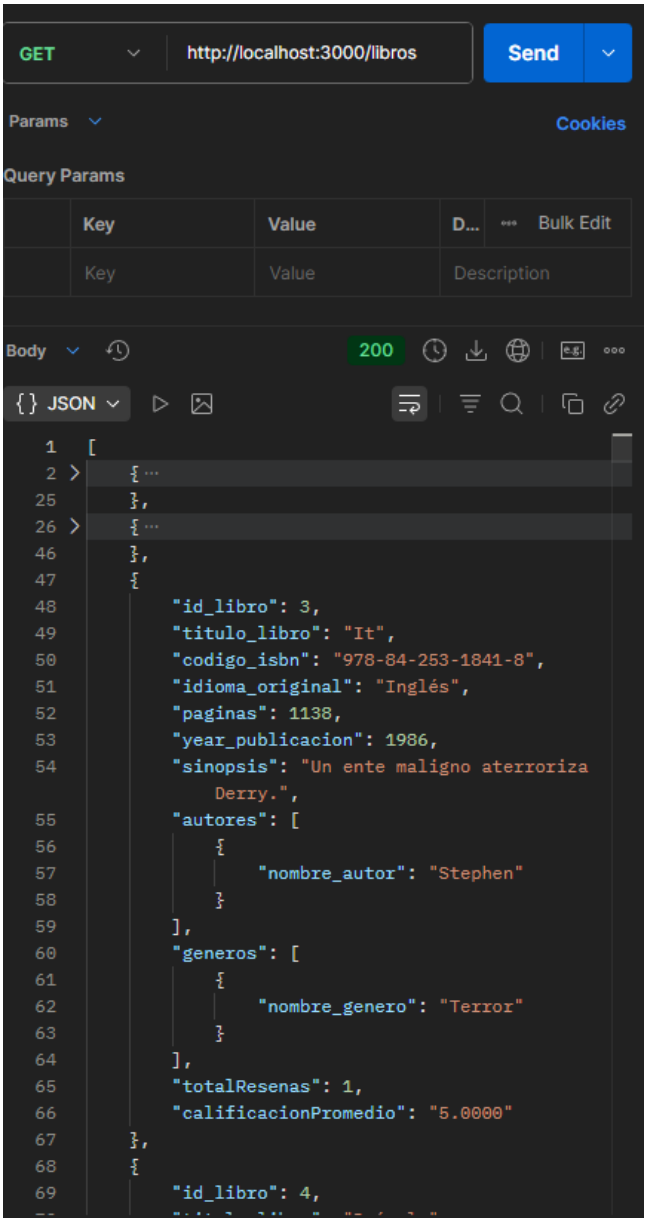
http://localhost:3000/resetAPI



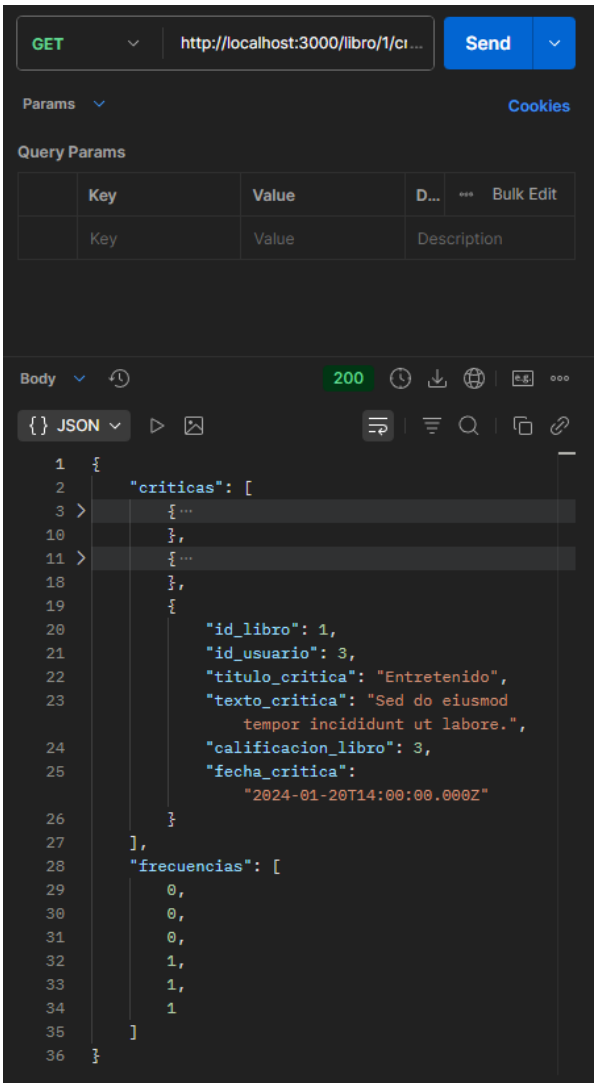
GET http://localhost:3000/libro/1



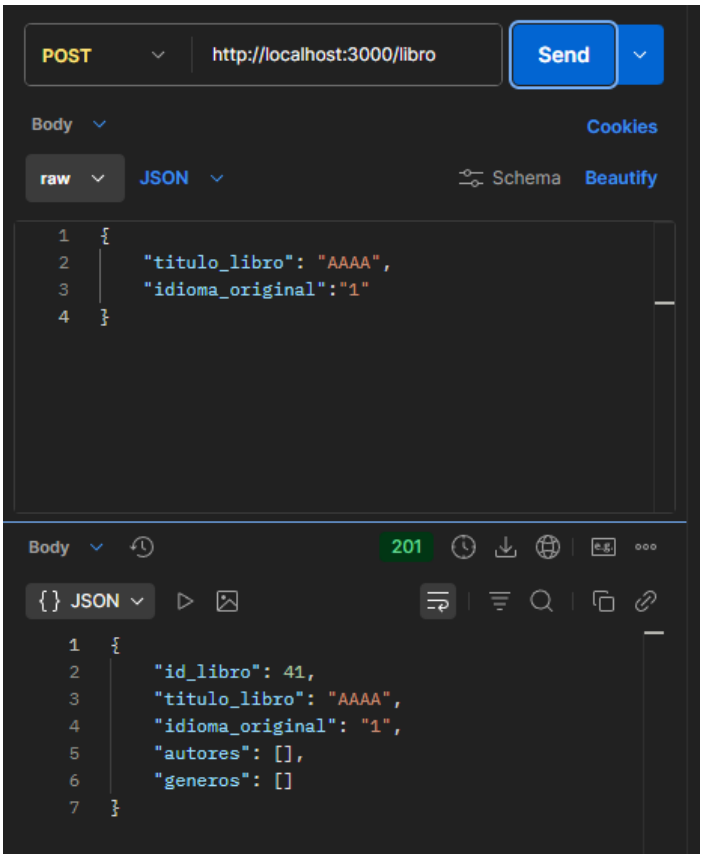
GET http://localhost:3000/libros



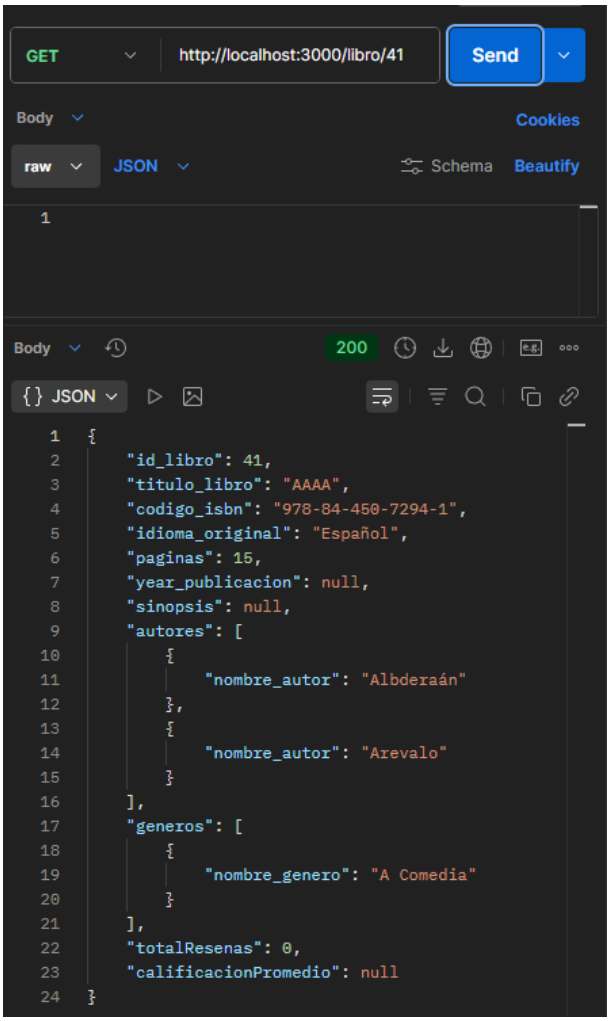
GET http://localhost:3000/libro/1/criticas



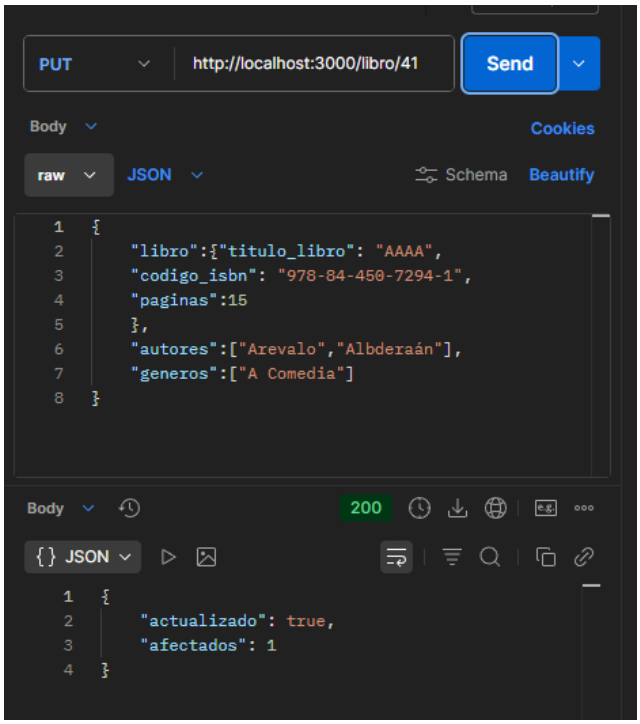
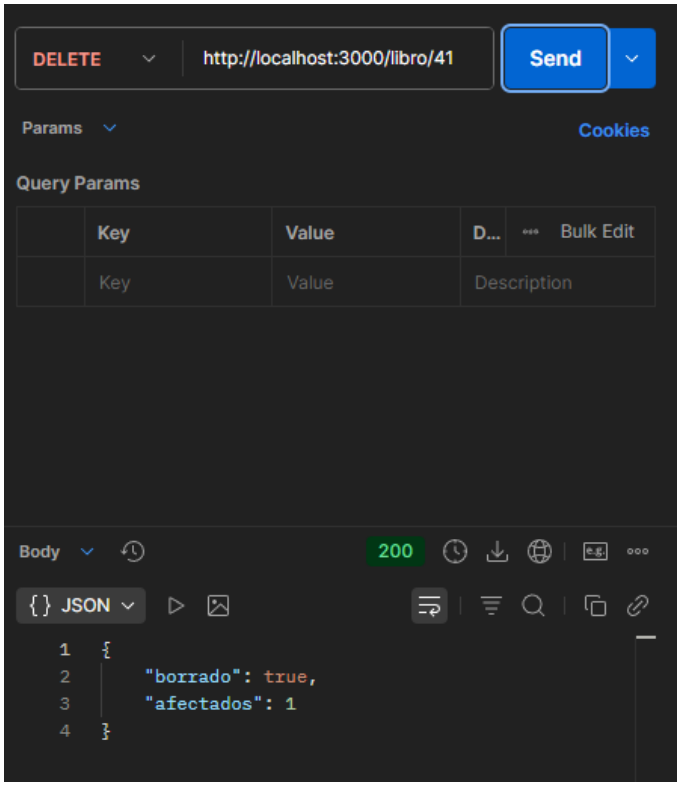
POST http://localhost:3000/libro



GET http://localhost:3000/libro/41



PUT http://localhost:3000/libro/41



DELETE http://localhost:3000/libro/41